

React 18 Cheatsheet

Hooks, state patterns and rendering rules on one printable sheet.

Core hooks

```
const [state, setState] = useState(initial);
useEffect(() => { /* setup */ return () => { /* cleanup */}; }, [deps]);
const ctx = useContext(ThemeContext);
const [v, dispatch] = useReducer(reducer, initialArg, init);
const ref = useRef(null); // mutable box, no re-render
const memo = useMemo(() => compute(a, b), [a, b]);
const cb = useCallback((x) => run(x), [run]);
```

Rules of hooks

- Call hooks only at the top level of a component or custom hook — never inside conditions or loops.
- Hooks must run in the same order on every render.
- Custom hooks are just functions whose name starts with use.

State updates

- State updates are asynchronous and batched inside event handlers and effects.
- Use the updater form when the next value depends on the previous one: `setCount(c => c + 1)`.
- Never mutate state objects or arrays — create new ones with spread or map.
- React 18 batches updates in promises, timeouts and native handlers too (automatic batching).

Effects

- Effects synchronise your component with something outside React (network, DOM, subscription).
- An empty dependency array runs once after mount; the cleanup runs on unmount.
- Missing dependencies cause stale values; the exhaustive-deps lint rule catches most cases.
- Prefer deriving values during render over storing them in state.

Performance

- React.memo skips re-rendering a component when its props are shallow-equal.
- Memoise expensive work with useMemo and stable callbacks with useCallback.
- Keep keys stable and unique in lists — index keys break reconciliation on reorder.
- Split context providers so a single value change does not re-render the whole tree.
- useDeferredValue and useTransition keep slow updates from blocking typing.

React 18 additions

- createRoot() replaces ReactDOM.render() and enables concurrent rendering.
- Automatic batching groups updates across async boundaries.
- Suspense works with data-fetching libraries; useId generates stable unique ids.
- StrictMode double-invokes effects in development to surface cleanup bugs.